

AI-Powered Assignment Feedback

1. Feature Name

AI-Powered Assignment Feedback

2. Purpose

- **Problem Solved:** Manually reviewing and commenting on every student submission is time-consuming for instructors.
 - **Why It Exists:** The feature gives instructors an AI-generated, structured first-pass review of a student's submission (TXT or PDF) — summary, grammar feedback, rubric coverage, missing topics, a suggested score, and a recommendation — which they can regenerate and use as advisory input.
 - **Role in Application:** It extends the Assignment & Submission workflow (Feature 8). It is explicitly **advisory**: the system prompt and disclaimer state the instructor remains the final evaluator. It is the only LLM/AI integration in the platform.
-

3. What the User Can Do

Instructor

- Generate AI feedback for a submission (TXT or PDF) in the classroom's assignment detail page.
- View the persisted AI feedback.
- Regenerate feedback (re-runs the LLM and overwrites the stored feedback).
- Students see the feedback only once the instructor publishes the evaluation (handled by the Assignment feature).

Student

- View AI feedback only after the evaluation is published (returned as part of `GetMySubmissionAsync`).
-

4. Feature Workflow

```
[Instructor clicks "Generate AI Feedback"]
→ POST /api/v1/submissions/{submissionId}/ai-feedback
→ AiFeedbackService.GenerateAsync(submissionId, userId)
→ Instructor role check (UserClassrooms Role == "Instructor")
→ ReadSubmissionTextAsync extracts text:
    .txt → read file directly
    .pdf → extract text via PdfPig (UglyToad)
→ AiFeedbackRequestDto built (assignment title/description + submission text)
→ PromptBuilder.BuildPrompt(request) → markdown-structured prompt
→ IAIProvider.SendAsync (NvidiaProvider → POST {BaseUrl}/chat/completions)
→ AiResponseParser.Parse(rawContent) → structured AiFeedbackResponseDto
```

- Persist/overwrite SubmissionAiFeedback row
- AI_FEEDBACK_GENERATED audit logged
- Return structured feedback

5. How It Works Internally

Provider Abstraction

`IAiProvider` (backend/Interfaces/Services/Ai/IAiProvider.cs) defines `SendAsync(AiProviderRequest)` returning `AiProviderResponse { Success, Content, ErrorMessage }`.

- **Active provider:** `NvidiaProvider` (registered in `Program.cs` via `AddHttpClient<IAiProvider, NvidiaProvider>()`).
- **Dormant provider:** `GeminiProvider` exists in `Services/Ai/` but is **not registered** in DI and is never used.
- Provider is chosen by DI registration only; `AiOptions.Provider` default string is `"Nvidia"` but is not used to switch providers at runtime.

NVIDIA API Call

`NvidiaProvider.SendAsync`:

- URL: `{BaseUrl}/chat/completions` (`BaseUrl` default `https://integrate.api.nvidia.com/v1`).
- Sends a chat completion request with `model`, a single user message containing the prompt, and `stream = false`.
- Bearer auth header set from `AiOptions.ApiKey`.
- Timeout enforced via `CancellationTokenSource(TimeSpan.FromSeconds(_options.TimeoutSeconds))` (default 60s).
- Error mapping: 401 → invalid key; 403 → denied; 429 → rate limited; 503 → service unavailable; empty content → checks `finish_reason` (`content_filter`, `length`); `TaskCanceledException` → timeout message.

Text Extraction (ReadSubmissionTextAsync)

- `.txt` → read the stored file stream to a string.
- `.pdf` → `UglyToad.PdfPig.PdfDocument.Open(stream)` then joins each page's `Text` with `\n`.
- Other extensions → `InvalidOperationException("AI feedback is not supported for {ext} files.")`.
- If no extractable text → `"The submission contains no extractable text."`

Prompt Construction

`PromptBuilder.BuildPrompt(request)` builds a single prompt:

- A system instruction stating the AI is an advisory assistant that "never assigns the official grade".
- Optional sections: Assignment Question, Assignment Description, Assignment Rubric (empty in the current code — rubric is always `string.Empty`).
- The student submission text.
- An output format contract requiring exact markdown headings: `## Summary`, `## Grammar & Language Feedback`, `## Rubric Coverage`, `## Missing Topics`, `## Suggested Score`, `## Overall Recommendation`.

Response Parsing (AiResponseParser.Parse)

- `ExtractSection` uses a regex with lookbehind/lookahead: `(?<^##\s*{Name}\s*\n)(.+?)(?=\n##\s|\n*$)` — matches the text between the `## Name` heading and the next heading (or end), with a fallback for `#` single-

hash headings.

- `ParseSuggestedScore` extracts `Percentage: N` and `Marks: N` via regex (case-insensitive), defaulting to 0.
- A fixed `Disclaimer` string is attached: *"This feedback is AI-generated and is intended only to assist the instructor. The final evaluation is determined solely by the instructor."*
- If `Summary` is empty after parsing → treated as failure (`"AI returned an incomplete response. Please try again."`).

Persistence & Regeneration

- `GenerateAsync` checks for an existing `SubmissionAiFeedback` row for the submission.
- Existing → overwrite all fields including `RawResponse` and reset `GeneratedAt` .
- None → insert a new row.
- Regeneration is detected by the existence of a prior row and reflected in the audit description (`"regenerated"` VS `"generated"`).

Retrieval & Visibility

- `GetAsync` returns the stored feedback to instructors.
- Students receive feedback only through `ClassroomService.GetMySubmissionAsync` , and only when `submission.PublishedAt != null` (see Feature 8).

6. Frontend Implementation

Layer	File Path	Responsibility
Component	<code>frontend/src/components/classroom/AiFeedbackSection/AiFeedbackSection.tsx</code>	Generate/regenerate button, status chips, structured feedback panel, disclaimer
Service	<code>frontend/src/services/classroomService.ts</code>	<code>getAiFeedback</code> , <code>generateAiFeedback</code>
Types	<code>frontend/src/types/classroom.types.ts</code>	<code>AiFeedbackResponse</code>
Page	<code>frontend/src/pages/classroom/AssignmentDetailPage.tsx</code>	Hosts the <code>AiFeedbackSection</code> for instructors

7. Backend Implementation

Layer	File Path	Responsibility
Controller	<code>backend/Controllers/SubmissionController.cs</code>	<code>GET/POST /submissions/{id}/ai-feedback</code>

Layer	File Path	Responsibility
Service	backend/Services/Ai/AiFeedbackService.cs	Orchestration, authorization, extraction, persistence, regeneration
Provider	backend/Services/Ai/NvidiaProvider.cs	NVIDIA chat-completions HTTP call + error mapping (active)
Provider	backend/Services/Ai/GeminiProvider.cs	Alternative provider (exists, not registered)
Prompt	backend/Services/Ai/PromptBuilder.cs	System instruction + structured output-format prompt
Parser	backend/Services/Ai/AiResponseParser.cs	Regex section extraction + score parsing
Config	backend/Configuration/AiOptions.cs	ApiKey, BaseUrl, Model, TimeoutSeconds
Interface	backend/Interfaces/Services/Ai/IAiProvider.cs	Provider contract + request/response types
Interface	backend/Interfaces/Services/Ai/IAiFeedbackService.cs	Generate/Get contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
SubmissionAiFeedbacks	Persisted structured AI feedback for a submission	FK <code>SubmissionId</code> → Submissions; stores Summary, GrammarFeedback, RubricCoverage, MissingTopics, SuggestedScorePercentage, SuggestedScoreMarks, OverallRecommendation, Disclaimer, RawResponse, GeneratedAt

- One feedback row per submission (regeneration overwrites rather than versioning).
- `RawResponse` stores the provider's full output for debugging.
- Added via migration `20260729102123_AddSubmissionAiFeedback.cs`.

9. Security & Authorization

- `GET / POST /ai-feedback` require authentication.
- Both generation and retrieval require the caller to be an **Instructor** of the submission's classroom (`UserClassrooms.Role == "Instructor"`); otherwise `UnauthorizedAccessException` → 403 Forbidden.
- Students cannot call these endpoints; they only receive published AI feedback through the assignment submission response.
- The API key lives in `AiOptions:ApiKey` loaded from `.env` (never committed); missing key produces a clear unavailable error.

10. Important Business Rules

1. **Instructor-only** generation and direct viewing.
 2. **Supported files:** TXT and PDF only for AI analysis; others rejected with a clear message.
 3. **Non-empty text required** — submissions without extractable text cannot be analyzed.
 4. **Advisory only:** the prompt and the stored disclaimer make clear the instructor assigns the official grade; suggested score is advisory.
 5. **Regeneration allowed** — re-runs the model and overwrites the stored feedback.
 6. **Student visibility** is gated by the Assignment publish flow (`PublishedAt`).
 7. **Missing API key or model failure** surfaces a user-friendly error rather than a crash.
-

11. Example of Internal Execution

Step 1: Instructor generates feedback

- **User Action:** Instructor opens the assignment detail and clicks "Generate AI Feedback" for submission 88.
- **Frontend:** `classroomService.generateAiFeedback(88)` → `POST /api/v1/submissions/88/ai-feedback`.
- **Controller → Service:** `AiFeedbackService.GenerateAsync(88, userId)`.
- **Authorization:** the caller is verified as an `Instructor` of the classroom.
- **Extraction:** the file is `.pdf` → `PdfPig` extracts the page text.
- **Prompt:** `PromptBuilder` builds the structured prompt (assignment title/description + submission text + output format).
- **Provider:** `NvidiaProvider` posts to `{BaseUrl}/chat/completions` with the model and prompt; the response content is returned.
- **Parse & persist:** `AiResponseParser` extracts Summary/Grammar/Rubric/Missing Topics/Score/Recommendation; a `SubmissionAiFeedback` row is created; `AI_FEEDBACK_GENERATED` audit logged.
- **Response:** the structured `AiFeedbackResponseDto` is returned and rendered by `AiFeedbackSection`.

Step 2: Regeneration

- The instructor clicks "Regenerate AI Feedback" — the same flow runs, an existing row is overwritten, and the audit message says "regenerated".

Step 3: Student visibility

- After the instructor publishes the evaluation, `GetMySubmissionAsync` attaches the stored AI feedback to the submission response the student receives.
-

12. Files Involved

Frontend

- `frontend/src/components/classroom/AiFeedbackSection/AiFeedbackSection.tsx`
- `frontend/src/services/classroomService.ts`

- `frontend/src/types/classroom.types.ts`
- `frontend/src/pages/classroom/AssignmentDetailPage.tsx`

Backend

- `backend/Controllers/SubmissionController.cs`
- `backend/Services/Ai/AiFeedbackService.cs`
- `backend/Services/Ai/NvidiaProvider.cs`
- `backend/Services/Ai/GeminiProvider.cs` (dormant)
- `backend/Services/Ai/PromptBuilder.cs`
- `backend/Services/Ai/AiResponseParser.cs`
- `backend/Configuration/AiOptions.cs`
- `backend/Interfaces/Services/Ai/IAiProvider.cs`
- `backend/Interfaces/Services/Ai/IAiFeedbackService.cs`

Database

- `backend/Entities/SubmissionAiFeedback.cs`
 - `backend/Migrations/20260729102123_AddSubmissionAiFeedback.cs`
-

13. Functionality

- Generate structured AI feedback for TXT/PDF submissions
- Extract text from TXT and PDF files (PdfPig)
- Build a structured, output-format-enforced prompt
- Call the NVIDIA LLM chat-completions API (with timeouts and error mapping)
- Parse the response into sections (summary, grammar, rubric coverage, missing topics, suggested score, recommendation)
- Persist and regenerate feedback per submission
- Instructor-only generation and viewing
- Students see AI feedback only after evaluation is published
- Audit logging of generation/regeneration